DESIGN AND ANALYSIS OF ALGORITHMS

SMART EMERGENCY RESOURCE ALLOCATOR

GUI-Based Algorithm Implementation • Assignment 2

Submitted by	_____
Name	_____
Roll Number	_____
Class / Semester	_____
Department	Computer Science and Engineering
College	_____
Academic Year	2026–2027

Submitted in partial fulfilment of the requirements for the course

DESIGN AND ANALYSIS OF ALGORITHMS

1. Project Title

Smart Emergency Resource Allocator using 0/1 Knapsack Dynamic Programming

2. Problem Statement

During emergencies such as floods, fires, or road accidents, responders may have limited carrying capacity in a vehicle or rescue kit. Each available resource has a weight and an importance (priority) value. The challenge is to choose a subset of resources that maximizes total priority without exceeding the available capacity. A resource is either selected completely or not selected; partial selection is not allowed.

3. Objectives

• Apply a DAA technique to a practical resource-allocation problem.
• Build a desktop GUI for entering resources and capacity.
• Find the optimal subset using 0/1 Knapsack dynamic programming.
• Display the DP table progressively to explain execution.
• Validate inputs and test normal, boundary, and invalid cases.

4. Proposed Solution and Application

The application accepts a list of emergency resources. Each resource has a name, integer weight, and non-negative priority value. The user specifies the maximum capacity. The solver constructs a dynamic programming table, computes the maximum achievable priority, and backtracks through the table to identify the selected resources. The GUI presents the result and allows the user to move forward and backward through the DP rows.

5. Algorithm: 0/1 Knapsack using Dynamic Programming

Let n be the number of resources and W be the maximum capacity. Let weight[i] and value[i] denote the weight and priority of resource i. Define dp[i][w] as the maximum priority obtainable using the first i resources with capacity w.
Base case: dp[0][w] = 0 for every capacity w.
Transition: If weight[i] > w, dp[i][w] = dp[i–1][w]. Otherwise:
dp[i][w] = max(dp[i–1][w], value[i] + dp[i–1][w – weight[i]])
The first option excludes the current resource; the second includes it.

6. Pseudocode

```
for i = 0 to n: dp[i][0..W] = 0
for i = 1 to n:
for w = 0 to W:
dp[i][w] = dp[i-1][w]
if weight[i] <= w:
dp[i][w] = max(dp[i-1][w], value[i] + dp[i-1][w-weight[i]])
Backtrack from dp[n][W] to recover chosen resources.
```

7. Complexity Analysis

Time Complexity: O(nW). There are (n+1)(W+1) DP states, each computed in constant time.
Space Complexity: O(nW). The complete two-dimensional table is retained for reconstruction and step-by-step visualization. Here n is the number of resources and W is the integer capacity.

8. GUI Design and Features

The interface is implemented using Python Tkinter. The left panel allows users to add and remove resources. The optimization panel accepts capacity and starts the solver. The result area displays maximum priority, selected weight, chosen resources, and complexity. The visualization panel displays DP rows progressively and provides Previous Step, Next Step, and Show Complete Table controls.

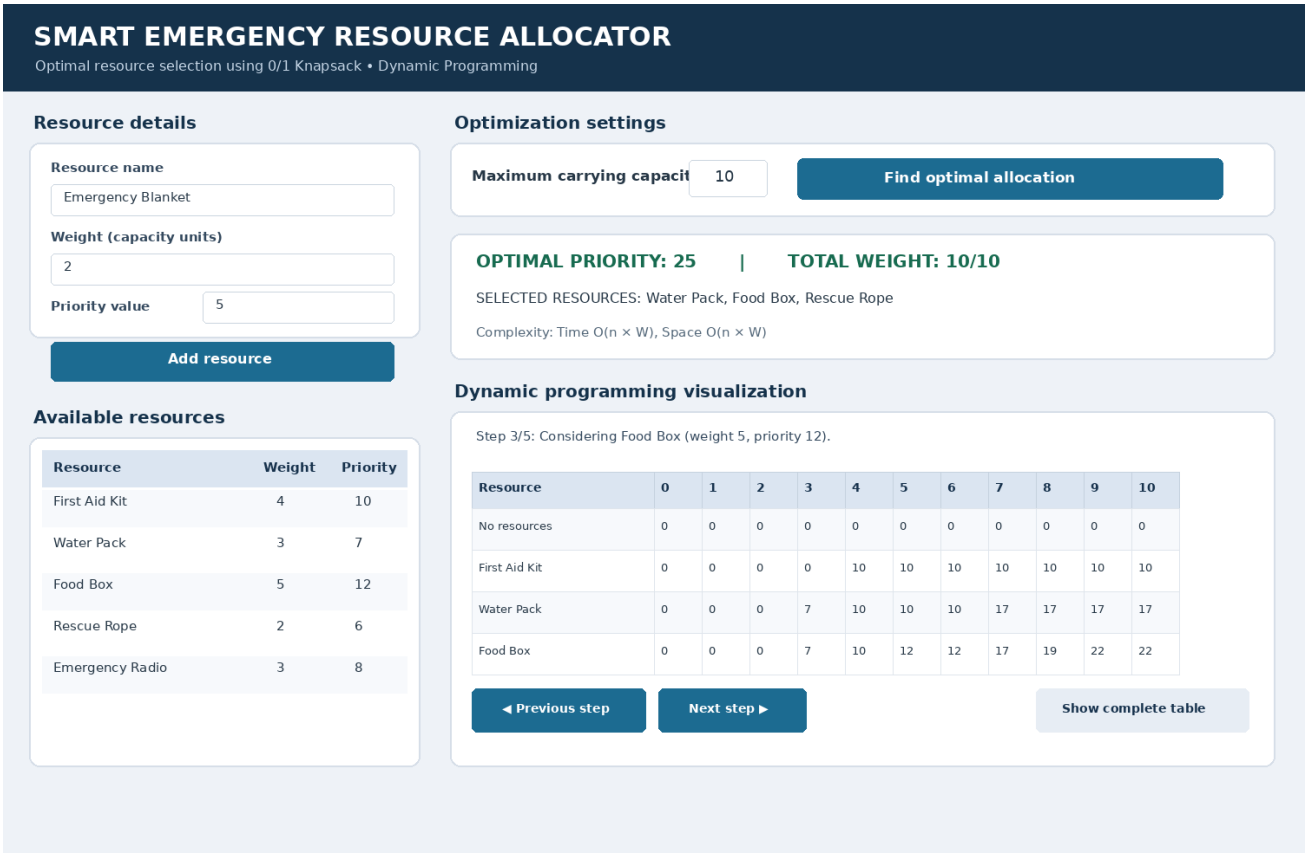


Figure 1. GUI preview showing resource entry, optimal result, and DP-table visualization.

9. Sample Input and Expected Result

Sample resources: First Aid Kit (weight 4, priority 10), Water Pack (3, 7), Food Box (5, 12), Rescue Rope (2, 6), Emergency Radio (3, 8). Capacity = 10. The optimal priority is **25**, achieved by selecting Water Pack, Food Box, and Rescue Rope (total weight 10, priority 7 + 12 + 6 = 25).

10. Testing and Results

The following tests cover normal operation, capacity boundaries, resource constraints, and invalid inputs. Run them in the GUI using the sample data or by entering the specified values.

Test	Input / Condition	Expected behaviour
1	Capacity 10; default resources	Optimal priority 25; DP table displayed
2	Capacity 0	No positive-weight resource selected; priority 0
3	Capacity 1; all weights greater than 1	No resource selected; priority 0
4	One resource: weight 3, value 9; capacity 3	Resource selected; priority 9
5	Resource weight greater than capacity	Resource excluded
6	Empty resource list	Validation message; algorithm does not run
7	Negative / non-integer capacity	Validation message displayed
8	Zero or negative resource weight	Resource entry rejected

11. Conclusion

The Smart Emergency Resource Allocator demonstrates how dynamic programming solves a practical optimization problem. It finds an optimal set of indivisible resources under a capacity constraint and makes the algorithm easier to understand through progressive DP-table visualization. It addresses algorithm implementation, GUI interaction, complexity

analysis, and boundary testing.

12. How to Run the Application

Install Python 3.9 or later with Tkinter. Extract the project folder, open a terminal in it, and run **python app.py**. No external Python packages are required. Refer to README.md for usage instructions.